

Software Architecture Document

Phase 1 — Version 1.0

Deeply Analytics

AI-Powered Microscopy Analysis — Nuclei Segmentation and Cell Counting

Course	Software Architecture Project
Phase	Phase 1 — Architecture Design & Initial Implementation
Deadline	04 May 2026
Document version	1.0
Issue date	02 May 2026
Architectural style	Layered REST API (FastAPI backend + React SPA)
Modelling approach	Kruchten 4+1 View Model (Use Case, Logical, Process)
Repository	github — project root (local)

Table of Contents

- 1. Introduction
 - 1.1 Purpose of this document
 - 1.2 Scope of Phase 1
 - 1.3 The 4+1 view model
 - 1.4 Definitions and acronyms
- 2. System Selection (Step 1)
 - 2.1 Target system: Deeply Analytics
 - 2.2 Problem statement
 - 2.3 System purpose and objectives
 - 2.4 Target users
 - 2.5 Core functionalities
 - 2.6 Architectural style: layered REST API
 - 2.7 Platform and technology stack
- 3. Use Case View (Step 2.1)
 - 3.1 Actors
 - 3.2 Use case diagram
 - 3.3 Use case specifications
 - 3.4 Partial UI code
- 4. Logical View (Step 2.2)
 - 4.1 Logical decomposition
 - 4.2 Class and component diagram
 - 4.3 Component responsibilities
 - 4.4 Key relationships
 - 4.5 Partial backend code
- 5. Process View (Step 2.3)
 - 5.1 Runtime processes
 - 5.2 Sequence diagram — analyze flow
 - 5.3 Activity diagram — analyze flow
 - 5.4 Health check flow
 - 5.5 Concurrency, blocking, and fallback behaviour
 - 5.6 Process-view code snippets
- 6. Phase 1 Implementation Status
- 7. References
- Appendix A — Repository layout

1. Introduction

1.1 Purpose of this document

This document is the Software Architecture Document (SAD), version 1, for the **Deeply Analytics** system — an AI-powered web application for nuclei segmentation and cell counting in microscopy images. It is delivered as the Phase 1 milestone of the Software Architecture Project (deadline 04.05.2026). The document describes the target system, the architectural style chosen, and the first three views of the Kruchten 4+1 model: the Use Case view, the Logical view, and the Process view.

1.2 Scope of Phase 1

Phase 1 is a partial implementation: the system is functional end-to-end for a single user uploading and analysing one image at a time. The backend exposes three REST endpoints (health check, analyse, file download); the frontend is a single-page React + Vite + TypeScript application with a four-step workflow (Upload → Segment → Count → Report). Persistence, authentication, multi-project support, asynchronous job queues, and the Development and Physical views of the 4+1 model are out of scope for Phase 1 and will be addressed in Phase 2.

1.3 The 4+1 view model

Kruchten's 4+1 model decomposes a software architecture into five concurrent views, each chosen to communicate with a specific audience. Phase 1 covers three of them, plus a unifying scenario set (the Use Case view that ties everything together).

- **Use Case view** — functional scenarios from the perspective of end users; ties the other views together.
- **Logical view** — static decomposition of the system into classes, modules, and components that satisfy the functional requirements.
- **Process view** — runtime behaviour, threads, interactions between processes, and how concurrency and blocking are handled.
- *Development view* (Phase 2) — how source code is organised into modules and packages.
- *Physical view* (Phase 2) — how software maps to deployment nodes (hosts, containers, networks).

1.4 Definitions and acronyms

Term	Meaning
SAD	Software Architecture Document.
MVP	Minimum Viable Product — smallest end-to-end version.
U-Net	Convolutional encoder-decoder architecture used for biomedical image segmentation (Ronneberger et al., 2015).
MoNuSeg	Multi-organ Nuclei Segmentation dataset; provides histopathology images and XML nucleus annotations.
CC labelling	Connected-component labelling — an OpenCV operation that groups adjacent foreground pixels into discrete objects, used here to count nuclei.
JWT	JSON Web Token — planned auth mechanism (Phase 2).
ORM	Object-Relational Mapper — SQLAlchemy/SQLModel.
SPA	Single-Page Application — the React frontend.

DTO	Data Transfer Object — here, Pydantic schemas.
-----	--

2. System Selection (Step 1)

2.1 Target system: Deeply Analytics

Deeply Analytics is a web-based platform that lets a biomedical researcher upload a microscopy image and receive, in seconds, a segmentation mask of the cell nuclei it contains, an overlay visualisation, and a cell-count estimate. The system is built on top of a trained U-Net (ResNet-18 encoder, *segmentation_models_pytorch*) and a connected-component post-processor that converts the binary mask into discrete nuclei objects.

2.2 Problem statement

Manual nuclei counting on histopathology slides is slow (several minutes per field of view), inconsistent between observers, and does not scale to studies that produce thousands of images. Existing offline scripts solve the throughput problem but require a Python environment, a GPU, and command-line literacy — none of which are typical for a wet-lab researcher. Deeply Analytics replaces the manual step with a browser-based workflow that anyone in the lab can use.

2.3 System purpose and objectives

- Reduce time spent on manual nuclei counting from minutes to seconds per image.
- Produce repeatable, operator-independent counts.
- Expose the AI pipeline through a usable REST API and single-page web interface.
- Provide a clean architectural baseline that can grow into a multi-user SaaS in Phase 2 without rewrites.
- Demonstrate sound application of the 4+1 architectural model and a layered REST architecture.

2.4 Target users

Actor	Role / context	Primary goals
Researcher	Biomedical or histopathology researcher analysing nuclei in tissue sections.	Upload an image, get a count and a mask, download the overlay for a paper.
Lab Technician	Wet-lab staff running routine cell counts.	Upload an image, view the result, repeat the workflow on a batch of slides.
Admin / Monitoring	Developer or operations role; in Phase 2, system administrator.	Check that the backend and the U-Net model are healthy via <code>/api/health</code> .

2.5 Core functionalities

ID	Functionality	Description	Status
F-1	Upload microscopy image	Accept PNG/JPG/TIFF up to 25 MB via multipart/form-data.	Implemented
F-2	Run nuclei segmentation	Run the U-Net model on the resized 256x256 image, applying sigmoid > 0.8 threshold.	Implemented
F-3	Count nuclei	Apply connected-component labelling on the binary mask to estimate cell count.	Implemented
F-4	Generate overlay visualisation	Composite the predicted mask onto the input as a red-tinted overlay.	Implemented

F-5	Persist artefacts	Save the input, the mask, and the overlay as PNGs in the storage layer.	Implemented (local FS)
F-6	Download artefacts	Serve PNG artefacts via GET /files/{filename} with path-traversal protection.	Implemented
F-7	Health monitoring	Report device, model load status, and operating mode via GET /api/health.	Implemented
F-8	Fallback demo path	If the U-Net checkpoint cannot load, switch to an Otsu-threshold demo so the UI is testable end-to-end.	Implemented
F-9	Multi-user persistence	Persist users, projects, jobs, and results in PostgreSQL.	Proposed (Phase 2)
F-10	Authentication	JWT-based login with role-based access control.	Proposed (Phase 2)
F-11	Asynchronous jobs	Run inference on a background worker so the API never blocks.	Proposed (Phase 2)

2.6 Architectural style: layered REST API

The system follows a **layered architecture** with a **REST API** as the integration boundary. Each layer has a single concern and depends only on the layer below it.

Layer	Responsibility	Implementation
Presentation (frontend)	Render UI, capture file uploads, display results.	React 18 + Vite + TypeScript SPA.
API (route layer)	HTTP transport, validation, error mapping.	FastAPI routers + Pydantic schemas (backend/main.py).
Service (orchestration)	Coordinate analysis workflow: load model, decode, predict, count, persist.	backend/services/analysis_service.py.
AI processing	Pure inference and image post-processing.	PyTorch U-Net + OpenCV + NumPy (src/infer.py, src/batch_count_refined.py).
Data / storage	Persist artefacts and (in Phase 2) structured records.	Local filesystem today; PostgreSQL + SQLAlchemy proposed.

2.7 Platform and technology stack

Layer	Technology	Version	Status
Frontend framework	React + React DOM	18.3.1	Implemented
Frontend tooling	Vite + TypeScript	5.4 / 5.5	Implemented
Backend framework	FastAPI	0.104.x	Implemented
ASGI server	Uvicorn	0.46.0	Implemented
Validation	Pydantic + pydantic-settings	2.12	Implemented
AI framework	PyTorch	2.10	Implemented
Segmentation models	segmentation_models_pytorch	0.5.0	Implemented
Image processing	OpenCV, NumPy, scikit-image	4.13 / 2.4 / 0.26	Implemented

Storage	Local filesystem (storage_root)	—	Implemented
Persistence (target)	PostgreSQL + SQLAlchemy + Alembic	16 / 0.0.14 / 0.24	Proposed
Authentication (target)	JWT (HS256)	—	Proposed
Async jobs (target)	FastAPI BackgroundTasks → Celery + Redis	—	Proposed

3. Use Case View (Step 2.1)

The Use Case view captures the system from the perspective of the actors that interact with it. It defines what the system must do, leaving how it does it to the Logical and Process views.

3.1 Actors

- **Researcher** — primary user. Uploads microscopy images, runs analyses, views results, downloads artefacts.
- **Lab Technician** — routine user. Uploads images and reads the cell count without going deep into the data.
- **Admin / Monitoring** — technical operator. Calls the health endpoint to check that the backend and the model are alive. In Phase 2 this actor will also manage users and projects.

3.2 Use case diagram

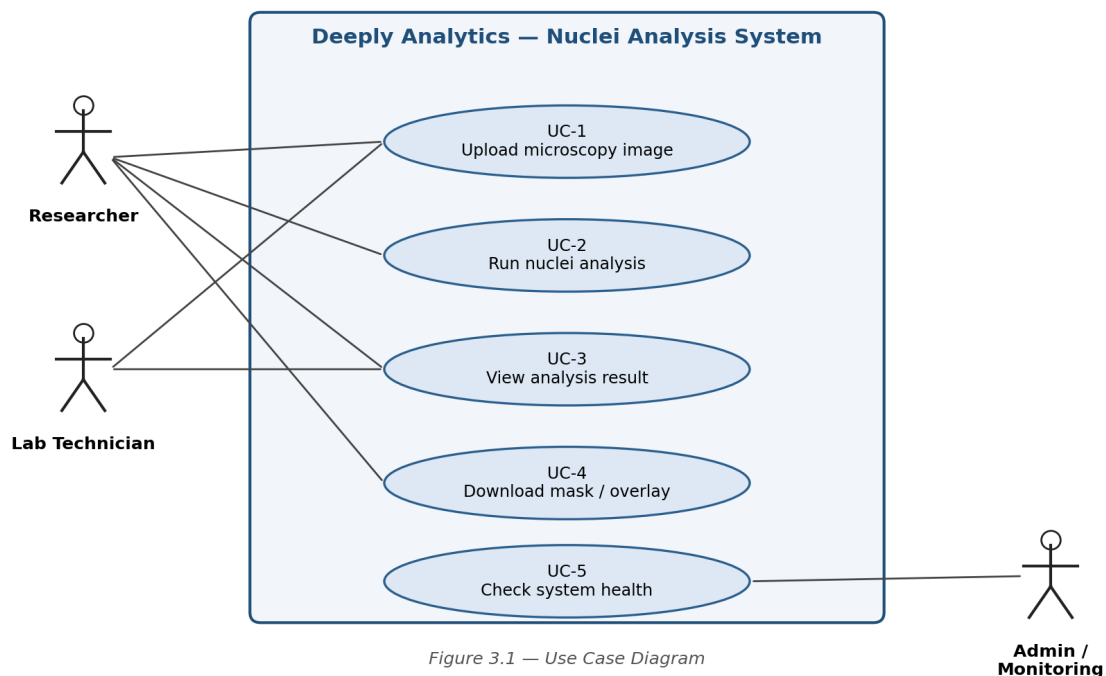


Figure 3.1 — Use Case Diagram

Figure 3.1 — Use Case Diagram

3.3 Use case specifications

UC-1 — Upload microscopy image

Field	Value
Actor(s)	Researcher, Lab Technician
Precondition	The user has the application open in a browser; the backend is reachable; the file is a supported image type (PNG, JPG, TIFF) and is under 25 MB.
Postcondition	A File reference is held in component state, ready for UC-2.
Alternative flow	If the user selects a non-image file, the browser file dialog filters it out; if a corrupt image is dropped, UC-2 will reject it with HTTP 400.

Main flow

1. The user drags an image onto the upload zone, or clicks to browse.
2. The frontend validates the file type and reads it into a File object.
3. The frontend renders a local preview of the image.
4. The Run-analysis button becomes enabled.

UC-2 — Run nuclei analysis

Field	Value
Actor(s)	Researcher, Lab Technician
Precondition	UC-1 has completed; the backend is reachable.
Postcondition	Three artefacts (input, mask, overlay) exist on disk; the response JSON contains the cell count and the URLs needed by UC-3 and UC-4.
Alternative flow	If the U-Net checkpoint is missing or fails to load, the service runs an Otsu-threshold fallback path and returns status 'ok-fallback'. If the uploaded file is not an image or is empty, the router returns HTTP 400. If any other error occurs, HTTP 500 with the exception message is returned.

Main flow

1. The user clicks Run analysis.
2. The frontend POSTs the file as multipart/form-data to /api/analyze.
3. The FastAPI router validates the MIME type and the size.
4. The AnalysisService lazy-loads the U-Net checkpoint on first call.
5. The service decodes and resizes the image to 256x256.
6. The U-Net produces logits, sigmoid is applied, threshold 0.8 yields a binary mask.
7. make_overlay() composites the mask onto the input as a red overlay.
8. count_nuclei_from_binary() counts connected components.
9. The service writes input, mask, and overlay PNGs to storage.
10. The router returns AnalysisResponse JSON containing job_id, cell_count, and three URLs.

UC-3 — View analysis result

Field	Value
Actor(s)	Researcher, Lab Technician
Precondition	UC-2 has returned a successful AnalysisResponse.
Postcondition	The user can read the count, see the segmentation, and decide whether to download.
Alternative flow	If the response is missing (network failure mid-call), the UI returns to the upload stage and displays a red error message.

Main flow

1. The frontend transitions the workflow stage from 'count' to 'report'.
2. ResultViewer renders a three-tile grid: Original, Mask, Overlay.

3. The cell count and the processing time are shown as metric cards.
4. A status badge indicates whether the U-Net or the fallback path produced the result.

UC-4 — Download artefacts

Field	Value
Actor(s)	Researcher
Precondition	UC-3 has rendered the results; the user wants to save the mask or the overlay.
Postcondition	The PNG is saved to the user's local filesystem.
Alternative flow	If the filename is invalid, the backend returns HTTP 400. If the file does not exist, HTTP 404.

Main flow

1. The user right-clicks the mask or overlay tile and chooses Save image as.
2. The browser issues GET /files/{filename} to the backend.
3. The router rejects any filename containing '/', '\\', or '..' (path-traversal guard).
4. If the file exists in the result directory, FastAPI returns it via FileResponse.

UC-5 — Check system health

Field	Value
Actor(s)	Admin / Monitoring
Precondition	The backend process is running.
Postcondition	The actor knows whether the backend, the device, and the model are operational.
Alternative flow	If the U-Net checkpoint is missing, mode = 'fallback-demo' and load_error carries the original exception text. The endpoint still returns HTTP 200 because the API itself is healthy — only the AI subsystem is degraded.

Main flow

1. The actor (or the React app on mount) issues GET /api/health.
2. The router calls AnalysisService.get_health(), which lazy-loads the model on first call.
3. The response contains: status, device (cpu/cuda), model_loaded, mode, load_error.

3.4 Partial UI code

The following excerpts show the UI surface that supports UC-1 through UC-3. Full source: *frontend/src/components/UploadPanel.tsx*, *ResultViewer.tsx*, and *api.ts*.

UploadPanel.tsx (drag-drop + run analysis)

```
export default function UploadPanel({
  file, previewUrl, busy, error, onFileSelected, onAnalyze,
}: Props) {
  const inputRef = useRef<HTMLInputElement>(null);
  const [dragging, setDragging] = useState(false);

  function handleDrop(e: DragEvent<HTMLDivElement>) {
    e.preventDefault();
```

```

    setDragging(false);
    const f = e.dataTransfer.files?.[0];
    if (f) onFileSelected(f);
  }

  return (
    <div className="card">
      <h3>1 . Upload</h3>
      <div className={`upload-zone ${dragging ? "dragging" : ""}`}
        onDrop={handleDrop}
        onDragOver={(e) => { e.preventDefault(); setDragging(true); }}
        onClick={() => inputRef.current?.click()}>
        <p>{file ? <strong>{file.name}</strong>
          : "Drop a microscopy image, or click to browse."}</p>
        <input ref={inputRef} type="file" accept="image/*"
          style={{ display: "none" }} onChange={handleChange} />
      </div>
      {previewUrl && <img src={previewUrl} alt="preview" />}
      <button className="btn" disabled={!file || busy} onClick={onAnalyze}>
        {busy ? "Analysing..." : "Run analysis"}
      </button>
      {error && <div className="error">{error}</div>}
    </div>
  );
}

```

api.ts (fetch wrapper)

```

export const API_BASE = "http://127.0.0.1:8080";

export async function getHealth(): Promise<HealthResponse> {
  const res = await fetch(`${API_BASE}/api/health`);
  if (!res.ok) throw new Error(`Health check failed (${res.status})`);
  return res.json();
}

export async function analyzeImage(file: File): Promise<AnalysisResponse> {
  const form = new FormData();
  form.append("file", file);
  const res = await fetch(`${API_BASE}/api/analyze`, {
    method: "POST",
    body: form,
  });
  if (!res.ok) {
    let detail = `Analysis failed (${res.status})`;
    try {
      const err = await res.json();
      if (err?.detail) detail = err.detail;
    } catch {}
    throw new Error(detail);
  }
  return res.json();
}

```

ResultViewer.tsx (results grid)

```

export default function ResultViewer({ result, busy }: Props) {
  if (!result) {
    return (
      <div className="card">
        <h3>2 . Result</h3>
        <p>{busy ? "Awaiting analysis output..."
          : "Upload an image and click Run analysis."}</p>
      </div>
    );
  }
  const isModel = result.metadata.mode === "model";

```

```

return (
  <div className="card">
    <h3>2 . Result . job {result.job_id}</h3>
    <div className="results-grid">
      <div className="result-tile"><h3>Original</h3>
        <img src={fileUrl(result.input_url)} alt="input" /></div>
      <div className="result-tile"><h3>Mask</h3>
        <img src={fileUrl(result.mask_url)} alt="mask" /></div>
      <div className="result-tile"><h3>Overlay</h3>
        <img src={fileUrl(result.overlay_url)} alt="overlay" /></div>
      </div>
      <MetricCard label="Estimated cell count" value={result.cell_count}
        caption={`${result.metadata.processing_ms} ms`} />
      <MetricCard label="Status" value={isModel ? "U-Net model" : "fallback demo"} />
    </div>
  );
}

```

4. Logical View (Step 2.2)

The Logical view describes the static decomposition of the system into modules and the relationships between them. It answers *what* the code is, not *when* it runs.

4.1 Logical decomposition

Deeply Analytics is divided into six logical components, plus a proposed persistence cluster (User, Project, AnalysisJob, AnalysisResult) that is designed but not yet wired:

- **React UI** — the SPA the user sees in the browser. Three components carry the workflow: UploadPanel, ResultViewer, and WorkflowSteps. A thin *api.ts* wraps fetch.
- **FastAPI App** — the HTTP boundary. Three routes (health, analyze, files) and CORS middleware.
- **Pydantic Schemas** — declarative DTOs that validate every request and response and generate OpenAPI.
- **AnalysisService** — the orchestrator. Owns the model singleton, decodes images, calls the U-Net or the fallback, runs counting, persists artefacts.
- **U-Net model** — the PyTorch network (*smp.Unet* with ResNet-18 encoder), loaded once and held as a module-level singleton.
- **OpenCV postprocessor** — image decode, resize, overlay rendering (*src.infer.make_overlay*) and connected-component counting (*src.batch_count_refined.count_nuclei_from_binary*).
- **Storage** — the filesystem under *backend/storage/* with *uploads/* and *results/* subfolders.
- **Persistence (proposed)** — SQLAlchemy tables User, Project, AnalysisJob, AnalysisResult; the schema exists in *backend/db/models/* but no router uses it yet.

4.2 Class and component diagram

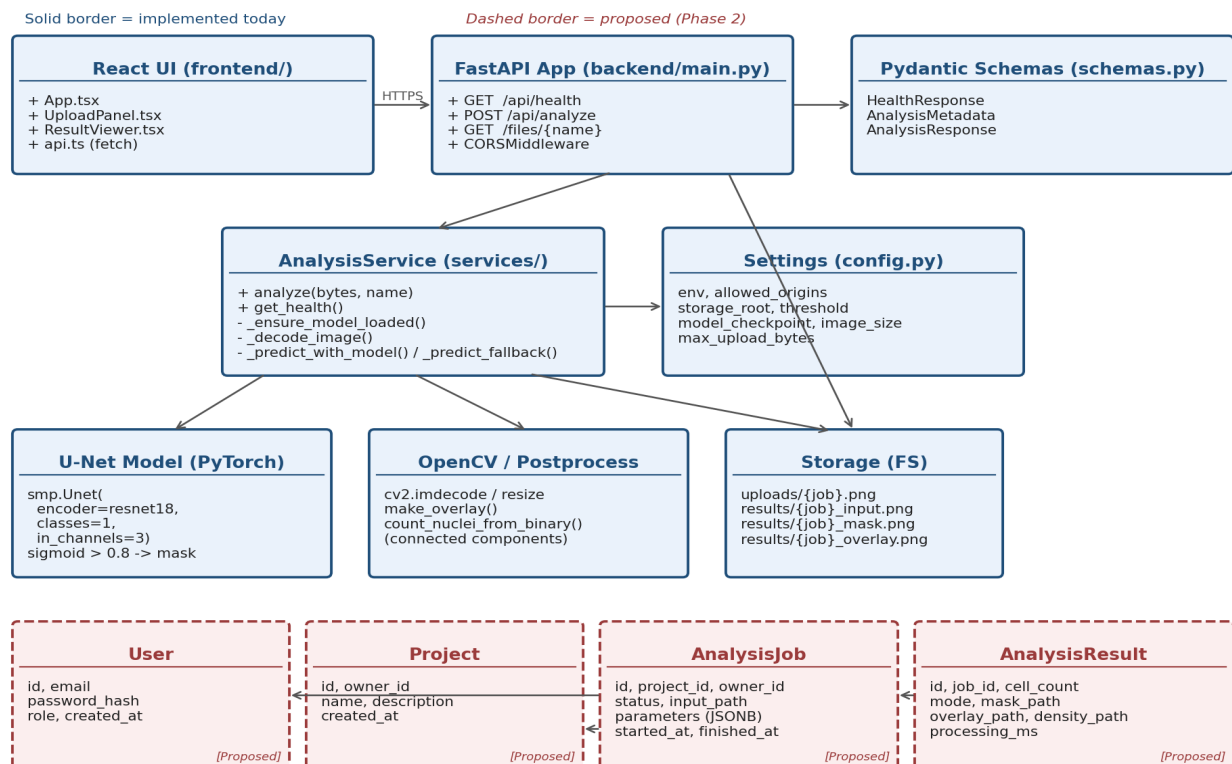


Figure 4.1 — Logical View: components and relationships

4.3 Component responsibilities

Component	Source file(s)	Responsibility
React UI	frontend/src/App.tsx, components/*.tsx	Owns workflow stage, renders panels, calls /api/health on mount, issues /api/analyze on user click.
api.ts	frontend/src/api.ts	Single source of truth for the API base URL and request shapes; wraps fetch and parses JSON.
FastAPI App	backend/main.py	Defines the three HTTP routes; applies CORS; maps service errors to HTTP 400 / 500.
Pydantic Schemas	backend/schemas.py	Defines HealthResponse, AnalysisMetadata, AnalysisResponse; ensures the contract with the frontend stays explicit.
AnalysisService	backend/services/analysis_service.py	Core workflow: lazy-load checkpoint, decode image, predict, count, persist, return AnalysisResult dataclass.
Settings	backend/config.py	Pydantic BaseSettings; reads env vars and .env; exposes derived paths (upload_dir, result_dir, checkpoint_path).
U-Net model	(loaded from outputs/checkpoints/best_model.pth)	ResNet-18 encoder + U-Net decoder, 1 output channel; produces logits that are sigmoid-thresholded at 0.8.
OpenCV postprocessor	src/infer.py, src/batch_count_refined.py	make_overlay() and count_nuclei_from_binary() are reused verbatim from the offline pipeline.
Storage	backend/storage/{uploads, results}	Holds the original upload (one per job) and three PNGs per completed analysis.
Persistence (proposed)	backend/db/models/*.py	SQLModel tables with foreign-key relationships; not yet exposed by any router.

4.4 Key relationships

- **Composition:** AnalysisService composes the U-Net model and the OpenCV postprocessor; their lifecycles are tied to the service module.
- **Dependency:** FastAPI routes depend on AnalysisService and on Pydantic schemas; the schemas depend on no internal module.
- **Uses:** AnalysisService uses Settings to discover paths and thresholds; it never reads environment variables directly.
- **Foreign keys (proposed):** AnalysisJob references User and Project; AnalysisResult references AnalysisJob with a unique constraint (one result per job).
- **HTTPS boundary:** the React UI talks to the FastAPI app over JSON; in Phase 2 a JWT will travel in an *HttpOnly* cookie.

4.5 Partial backend code

backend/main.py — FastAPI routes

```
app = FastAPI(title="Nuclei MVP API", version="0.1.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.allowed_origins,
    allow_credentials=False,
    allow_methods=["*"],
```

```

        allow_headers=["*"],
    )

@app.get("/api/health", response_model=HealthResponse)
def health() -> HealthResponse:
    return HealthResponse(**analysis_service.get_health())

@app.post("/api/analyze", response_model=AnalysisResponse)
async def analyze(file: UploadFile = File(...)) -> AnalysisResponse:
    if not (file.content_type or "").startswith("image/"):
        raise HTTPException(status_code=400, detail="File must be an image.")
    payload = await file.read()
    if not payload:
        raise HTTPException(status_code=400, detail="Empty upload.")
    try:
        result = analysis_service.analyze(payload, file.filename or "upload.png")
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Analysis failed: {e}")
    return AnalysisResponse(**analysis_service.result_to_dict(result))

@app.get("/files/{filename}")
def get_file(filename: str):
    if "/" in filename or "\\" in filename or "." in filename:
        raise HTTPException(status_code=400, detail="Invalid filename.")
    target = RESULT_DIR / filename
    if not target.exists() or not target.is_file():
        raise HTTPException(status_code=404, detail="File not found.")
    return FileResponse(target)

```

backend/schemas.py — Pydantic DTOs

```

class HealthResponse(BaseModel):
    status: str
    device: str
    model_loaded: bool
    mode: str = Field(description='"model" | "fallback-demo" | "uninitialised"')
    load_error: Optional[str] = None

class AnalysisMetadata(BaseModel):
    original_filename: str
    mode: str
    threshold: Optional[float] = None
    min_area: int
    image_size: int
    processing_ms: int
    device: str

class AnalysisResponse(BaseModel):
    job_id: str
    status: str
    message: str
    cell_count: int
    input_url: str
    mask_url: str
    overlay_url: str
    metadata: AnalysisMetadata

```

backend/db/models/analysis_job.py — proposed persistence

```

class JobStatus(str, Enum):
    pending = "pending"
    processing = "processing"
    completed = "completed"
    failed = "failed"

```



```

class AnalysisJob(SQLModel, table=True):
    __tablename__ = "analysis_jobs"

    id: Optional[int] = Field(default=None, primary_key=True)
    project_id: int = Field(foreign_key="projects.id", index=True)
    owner_id: int = Field(foreign_key="users.id", index=True)
    status: str = Field(default=JobStatus.pending.value, max_length=16, index=True)
    input_path: str = Field(max_length=500)
    parameters: dict[str, Any] = Field(
        default_factory=dict,
        sa_column=Column(JSONB, nullable=False, server_default("{}")),
    )
    started_at: Optional[datetime] = None
    finished_at: Optional[datetime] = None
    error_message: Optional[str] = None
    created_at: datetime = Field(
        default_factory=lambda: datetime.now(timezone.utc), nullable=False
    )

```

5. Process View (Step 2.3)

The Process view explains how the system behaves at runtime: which processes exist, how they communicate, where they block, and how concurrency is handled.

5.1 Runtime processes

In Phase 1 the backend runs as a single Uvicorn worker process. Inside that process, FastAPI dispatches HTTP requests to async handlers; the analysis handler is declared *async* but delegates to a synchronous service (the U-Net forward pass is synchronous PyTorch). The U-Net is loaded once on first use and kept as a module-level singleton inside *analysis_service.py*; subsequent requests reuse the loaded model. The frontend runs in the user's browser as a React SPA served by the Vite dev server in development; in Phase 2 it is built to static assets and served by a CDN. No background workers, queues, or long-running tasks exist in Phase 1.

5.2 Sequence diagram — analyze flow

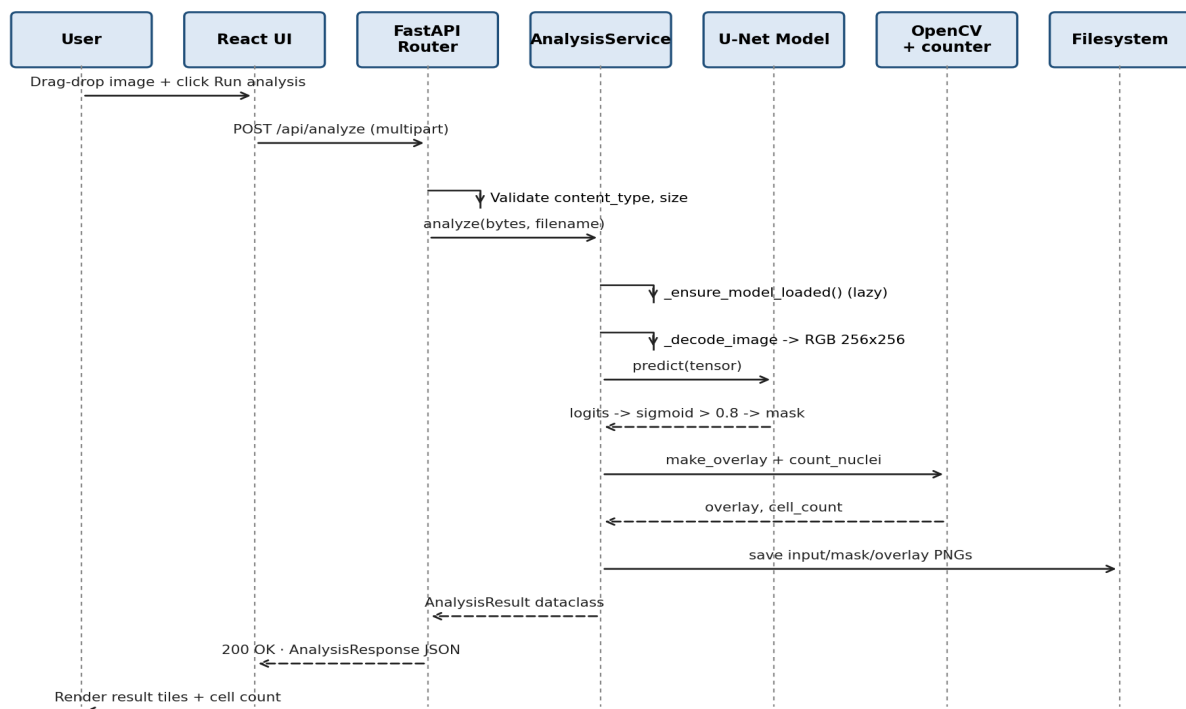


Figure 5.1 — Sequence Diagram: POST /api/analyze

Figure 5.1 — Sequence Diagram: POST /api/analyze

5.3 Activity diagram — analyze flow

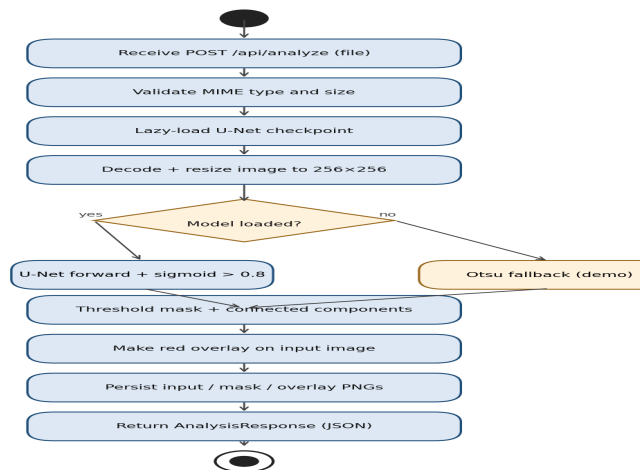


Figure 5.2 — Activity Diagram: Analyze workflow

Figure 5.2 — Activity Diagram: analyze workflow

5.4 Health check flow

GET /api/health is a short-circuit synchronous flow: the route calls `analysis_service.get_health()`, which calls `_ensure_model_loaded()` (no-op after the first call) and returns a dict with the device, the `model_loaded` boolean, the operating mode, and any load error. The React app calls this endpoint on mount so that the header badge can show *backend: model · cpu* or *backend: fallback-demo* before the user even attempts to analyse an image.

5.5 Concurrency, blocking, and fallback behaviour

- **Single-process MVP.** Multiple concurrent requests are accepted but the U-Net forward pass holds the Python GIL, so two simultaneous analyses are serialised in practice. This is acceptable for a single-user MVP; in Phase 2 a Celery + Redis worker pool will run inference jobs in parallel processes.
- **Lazy initialisation.** The U-Net is loaded on the first request that needs it. The first request after a cold start therefore takes a few seconds longer than subsequent requests; this is documented in the health endpoint by the transition *uninitialised* → *model*.
- **Graceful fallback.** If the checkpoint file is missing, corrupted, or incompatible with the current `segmentation_models_pytorch` version, `_ensure_model_loaded()` catches the exception, stores it in `_load_error`, and switches the service into *fallback-demo* mode (Otsu threshold + morphological opening). The API contract is preserved; only the response status changes from *ok* to *ok-fallback*.
- **Bounded inputs.** The request body is rejected before it is touched if the content type is not *image/**, if the upload is empty, or if it exceeds `max_upload_bytes` (25 MB). This bounds the per-request memory footprint.
- **Path-traversal protection.** GET /files/{filename} rejects any name containing /, \, or .. before resolving the path. This prevents the file-download route from leaking files outside the result directory.

5.6 Process-view code snippets

AnalysisService.analyze() — orchestration body

```

def analyze(image_bytes: bytes, original_filename: str) -> AnalysisResult:
    if len(image_bytes) > MAX_UPLOAD_BYTES:
        raise ValueError(f"Upload exceeds {MAX_UPLOAD_BYTES} bytes.")

    _ensure_model_loaded()

    job_id = uuid.uuid4().hex[:12]
  
```

```

suffix = Path(original_filename).suffix.lower() or ".png"
if suffix not in {".png", ".jpg", ".jpeg", ".tif", ".tiff", ".bmp"}:
    suffix = ".png"

upload_path = UPLOAD_DIR / f"{job_id}{suffix}"
upload_path.write_bytes(image_bytes)

image_rgb = _decode_image(image_bytes)

started = time.perf_counter()
if _mode == "model":
    mask = _predict_with_model(image_rgb)
    used_min_area = MIN_AREA
else:
    mask = _predict_fallback(image_rgb)
    used_min_area = 20
overlay = make_overlay(image_rgb, mask)
cell_count = count_nuclei_from_binary(mask, min_area=used_min_area)
elapsed_ms = int((time.perf_counter() - started) * 1000)

cv2.imwrite(str(RESULT_DIR / f"{job_id}_input.png"),
            cv2.cvtColor(image_rgb, cv2.COLOR_RGB2BGR))
cv2.imwrite(str(RESULT_DIR / f"{job_id}_mask.png"), mask)
cv2.imwrite(str(RESULT_DIR / f"{job_id}_overlay.png"),
            cv2.cvtColor(overlay, cv2.COLOR_RGB2BGR))

return AnalysisResult(
    job_id=job_id,
    status="ok" if _mode == "model" else "ok-fallback",
    message=(
        "Analysis complete (U-Net inference)." if _mode == "model"
        else "Analysis complete (fallback demo mode)."),
    cell_count=int(cell_count),
    input_url=f"/files/{job_id}_input.png",
    mask_url=f"/files/{job_id}_mask.png",
    overlay_url=f"/files/{job_id}_overlay.png",
    metadata={
        "original_filename": original_filename,
        "mode": _mode,
        "threshold": THRESHOLD if _mode == "model" else None,
        "min_area": used_min_area,
        "image_size": IMAGE_SIZE,
        "processing_ms": elapsed_ms,
        "device": _device or "cpu",
    },
)

```

ensure_model_loaded() — lazy init pattern

```

def _ensure_model_loaded() -> None:
    """Load the U-Net checkpoint once. On failure, switch to fallback-demo."""
    global _model, _device, _mode, _load_error
    if _mode != "uninitialised":
        return
    try:
        import torch
        import segmentation_models_pytorch as smp
        _device = "cuda" if torch.cuda.is_available() else "cpu"
        model = smp.Unet(
            encoder_name="resnet18",
            encoder_weights=None,
            in_channels=3,
            classes=1,
        ).to(_device)
        if not CHECKPOINT_PATH.exists():
            raise FileNotFoundError(f"Checkpoint not found: {CHECKPOINT_PATH}")
        state = torch.load(CHECKPOINT_PATH, map_location=_device)
        model.load_state_dict(state)
    
```

```
model.eval()
_model = model
_mode = "model"
except Exception as e:
    _load_error = f"{type(e).__name__}: {e}"
    _mode = "fallback-demo"
    _model = None
```

6. Phase 1 Implementation Status

This section records what was actually built for the Phase 1 milestone, separately from what is designed and documented in Sections 3 to 5. It is intended to be honest about which parts of the architecture are running today and which are planned.

6.1 What is implemented

- FastAPI backend with three working routes: `/api/health`, `/api/analyze`, `/files/{filename}`.
- PyTorch U-Net inference (ResNet-18 encoder), with a trained checkpoint at `outputs/checkpoints/best_model.pth`.
- OpenCV-based connected-component nuclei counter and red-tinted overlay generator, reused from the offline pipeline in `src/`.
- Otsu-threshold fallback path that keeps the API contract usable when the checkpoint cannot be loaded.
- React 18 + Vite + TypeScript single-page app with drag-drop upload, four-step workflow indicator, and result tile grid (Original / Mask / Overlay).
- Local filesystem storage layout with deterministic filenames keyed by a 12-character `job_id`.
- Pydantic settings loaded from `.env` with safe defaults so the app boots without any environment configuration.

6.2 What is proposed for Phase 2

- PostgreSQL persistence: User, Project, AnalysisJob, AnalysisResult tables (the SQLAlchemy schema is already drafted in `backend/db/models/`).
- Authentication and authorisation: JWT issuance, login and refresh routes, role-based access control, and ownership checks on every project-scoped resource.
- Asynchronous job processing: dispatch the analyse step to a Celery + Redis worker pool so the HTTP request returns immediately with a job ID and the frontend polls for status.
- Multi-page dashboard: login, project list, history, morphology features, density heatmaps, and exports.
- S3-compatible object storage replacing the local `storage/` tree for production deployments.
- Development view and Physical view added to the SAD (version 2).

6.3 Verified end-to-end

Twelve sample analyses are present in `backend/storage/results/`, each producing the expected trio of `{job_id}_input.png`, `{job_id}_mask.png`, and `{job_id}_overlay.png`. They demonstrate that the analyse flow described in Sections 5.2 and 5.3 runs to completion against real microscopy inputs and that the storage naming convention from Section 4 is honoured.

7. References

- Kruchten, P. (1995). *The 4+1 View Model of Architecture*. IEEE Software, 12(6), 42-50.
- Ronneberger, O., Fischer, P., & Brox, T. (2015). *U-Net: Convolutional Networks for Biomedical Image Segmentation*. MICCAI, 234-241.
- Iakubovskii, P. (2019). *Segmentation Models PyTorch*. GitHub: qubvel/segmentation_models.pytorch.
- Paszke, A. et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. NeurIPS 32.
- Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools.
- Tiangolo, S. (2018-). *FastAPI Documentation*. <https://fastapi.tiangolo.com/>
- Colvin, S. (2017-). *Pydantic Documentation*. <https://docs.pydantic.dev/>
- Tiangolo, S. (2021-). *SQLModel Documentation*. <https://sqlmodel.tiangolo.com/>
- Bayer, M. (2010-). *Alembic Documentation*. <https://alembic.sqlalchemy.org/>

Appendix A — Repository layout

```
project/
+-- backend/
|   +-- main.py           FastAPI app, 3 routes
|   +-- config.py         Pydantic Settings
|   +-- schemas.py        Pydantic response models
|   +-- services/
|       \-- analysis_service.py  Orchestration + U-Net wrapper
|   +-- db/
|       +-- base.py       SQLAlchemy metadata
|       +-- session.py    Lazy engine + get_session()
|       \-- models/
|           +-- user.py
|           +-- project.py
|           +-- analysis_job.py
|           \-- analysis_result.py
|   +-- alembic/          Migration scripts
|   \-- storage/
|       +-- uploads/      One file per upload
|       \-- results/      {job}_input.png / _mask.png / _overlay.png
+-- frontend/
|   +-- index.html
|   +-- package.json      React 18, Vite 5, TypeScript
|   +-- vite.config.ts
|   \-- src/
|       +-- main.tsx      React 18 root
|       +-- App.tsx       Workflow orchestrator
|       +-- api.ts        fetch wrapper + DTOs
|       +-- styles.css    Design tokens
|       \-- components/
|           +-- UploadPanel.tsx
|           +-- ResultViewer.tsx
|           +-- WorkflowSteps.tsx
|           \-- MetricCard.tsx
+-- src/                  Offline AI pipeline (training, batch inference)
|   +-- train.py
|   +-- infer.py          make_overlay()
|   +-- batch_count_refined.py  count_nuclei_from_binary()
|   +-- ... (dataset, metrics, postprocess, density, morphology)
+-- data/
|   +-- raw/
|   +-- processed/
|   \-- splits/          train.csv / val.csv / test.csv / all.csv
+-- outputs/
|   +-- checkpoints/best_model.pth    Trained U-Net
|   +-- counting_batch_refined_xmlgt/ Batch inference outputs
|   +-- density_maps/
|   \-- morphology/
+-- docs/                Architecture and design documents
+-- scripts/
|   \-- build_sad_phase1.py    Builds this PDF
+-- requirements.txt
+-- .env.example
\-- README.md
```

Appendix B — Configuration reference

All settings are loaded by *backend/config.py* via *pydantic-settings*. Defaults are safe so the app boots without an *.env* file.

Setting	Default	Purpose
env	dev	Environment selector (dev / staging / prod).

allowed_origins	[localhost:5173, 127.0.0.1:5173]	CORS allow-list (Vite dev server).
database_url	(unset)	Phase 2: PostgreSQL DSN.
jwt_secret	(unset)	Phase 2: HS256 signing key.
jwt_alg	HS256	Phase 2: JWT algorithm.
jwt_ttl_seconds	3600	Phase 2: token lifetime.
storage_root	{ROOT}/backend/storage	Filesystem root for uploads and results.
model_checkpoint	outputs/checkpoints/best_model.pth	Path to the trained U-Net weights.
image_size	256	Fixed input/output resolution; tied to the trained network.
threshold	0.8	Sigmoid threshold for the binary mask.
min_area	1	Minimum connected-component size to count (model path).
max_upload_bytes	26214400 (25 MB)	Hard cap on a single upload.
rate_limit_*	10 / 30 / 60 / minute	Phase 2: per-route rate limits.